



1.2-Disciplinas de la Ingeniería de Software

Ing. Mónica Colombo
Ciclo Lectivo: 2026

¿Por qué seguimos con dificultades para medir el avance mientras se desarrolla y mantiene el software?

¿Por qué son tan altos los costos de desarrollo?

¿Por qué no podemos detectar todos los errores antes de entregar el software a nuestros clientes?

¿Por qué dedicamos tanto tiempo y esfuerzo a mantener los programas existentes?

¿Por qué se requiere tanto tiempo para terminar el software?



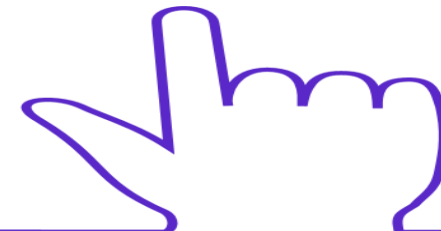
Contenidos: 1.2-Disciplinas de la Ingeniería de Sw

1. Entendiendo la ingeniería de software
2. Disciplinas relacionadas
3. Siete principios de la ingeniería de software
4. Código Limpio y código espagueti (A EVALUAR EN CLASES, ESTUDIAR)
5. Ingeniería de software como disciplina profesional
6. Desarrollo Global de software
7. Beneficios, desafíos y problemas del DGS
8. Modelo de provisión y la globalización de servicios



BIBLIOGRAFIA

- **CÓDIGO LIMPIO DE ROBERT C. MARTIN**
- **ARQUITECTURA LIMPIA” DE ROBERT C. MARTIN**
- INGENIERIA DE SOFTWARE” 6TA. ED. IAN SOMMERVILLE.- ED. ADISON WESLEY (2002)
- “INGENIERIA DEL SOFTWARE: UN ENFOQUE PRACTICO” 7MA ED.DE ROGER S.PRESSMAN ED. MCGRAWHILL (2010)
- “FABRICAS DE SOFTWARE: EXPERIENCIAS, TECNOLOGIAS Y ORGANIZACIÓN” DE MARIO PIATTINI Y JAVIER GARZAS PARRA- ED ALFAOMEGA RA-MA (2007)
- “CALIDAD EN EL DESARROLLO Y MANTENIMIENTO DEL SOFTWARE” DE MARIO PIATTINI Y FELIX GARCIA- ED ALFAOMEGA RA-MA (2003)
- “INGENIERIA DE SOFTWARE, UNA PERSPECTIVA ORIENTADA A OBJETOS” DE ERIC BRAUDE, ED ALFAOMEGA (2005)
- “INGENIERIA DEL SOFTWARE, UN ENFOQUE DESDE LA GUIA SWEBOK” DE SALVADOR SANCHEZ, MIGUEL SICILIA Y DANIEL RODRIGUEZ- ED ALFAOMEGA (2011)



1- Entendiendo la Ingeniería de Sw

La ingeniería de software cubre un marco muy amplio. Hay que entender esto como la posibilidad de que enmarque **varios objetivos a tener en cuenta** cuando queremos implementar u optar por un servicio de ingeniería de software, buscando que se cumplan algunos de ellos, **ya que las empresas que contratan este servicio no requieren el mismo tipo de proyecto:**



2-Disciplinas relacionadas

Las disciplinas del software son comúnmente utilizadas para lograr convertirse en un ingeniero en software calificado para realizar cualquier cosa que se cruce en su ámbito profesional.

Modelado de Negocios

Es entender los problemas que se desean solucionar por parte del usuario y asegurar que se entiendan que problema se les está presentando.

Requerimientos

Establecer acuerdos entre los usuarios acerca de que es lo que debe de hacer el sistema.

Desde aquí se empieza la planeación del sistema y al usuario se le estima los costos el tiempo de desarrollo del sistema

Análisis y Diseño

Desarrollar una arquitectura robusta para que se adapte a cualquier lugar de trabajo, así como también un desempeño esperando.

Pruebas

Encontrar los problemas que se presenten el software, también como la calidad de este, calificar o verificar que se haya hecho de acuerdo a lo planeado.



Administración y configuración del cambio

Controlar los cambios y mantener la integridad de los productos que incluye el proyecto.

Administración de proyectos

Provee guías para los ambientes de trabajo, como lo son la planeación, soporte, ejecución y monitoreo del proyecto

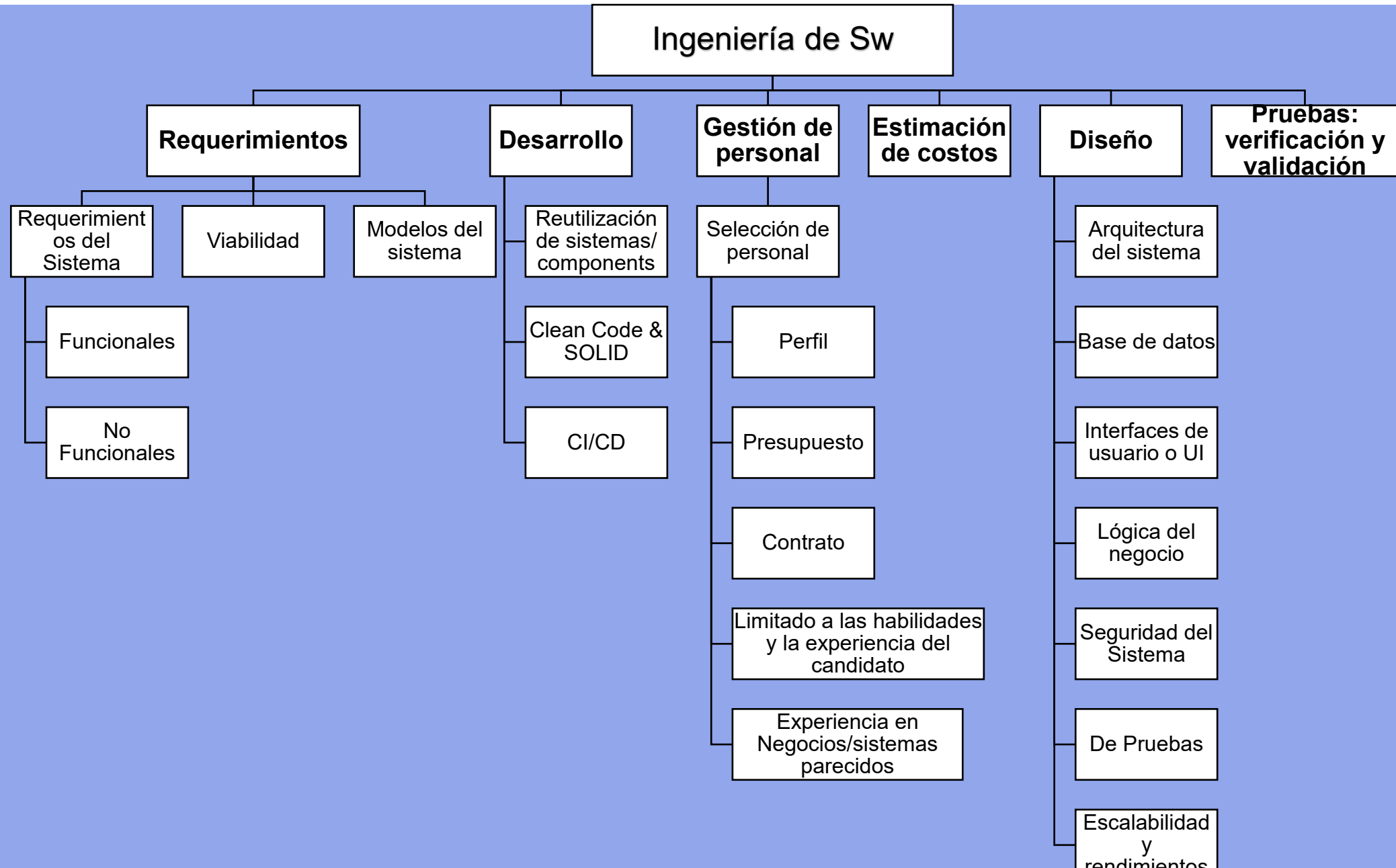
Ambientes

Desarrollar el apoyo al proyecto y su despliegue

Implementación

Codificar el sistema teniendo en cuenta la organización, componentes tales como: archivos ejecutables y de código fuente, y todo otro tipo de archivo que sean necesarios para el desarrollo, implementación, despliegue del sistema y mantenimiento

2-Disciplinas relacionadas



3-Los 7 principios de la ingeniería de software

Un principio es una ley importante y es requerida en un sistema de pensamiento.

1.La razón que exista todo

2.Mantenerlo sencillo

3.Mantener la vision

4.Otros consumirán lo que se produce

5.Abrazar al futuro

6.Planear por anticipado la reutilización

7.Pensar

3-Siete principios de la ingeniería de software(1/4)

La práctica de la ingeniería de software se centra en estos **siete principios**:

1er. principio: La razón que exista todo

- Esto responde a la pregunta ¿Esto agrega valor real al sistema?, cada paso o actividad antes de ponerla en práctica, se tiene que tomar en cuenta **si agrega valor al sistema**.

Ej: Antes de desarrollar una nueva aplicación, se realiza un análisis para determinar si realmente soluciona un problema o si ya existen soluciones adecuadas en el mercado.

2do. principio: Mantenerlo sencillo.

- Todo sistema **debe ser diseñado para ser sencillo**, lo más posible. Esto lo hace más fácil de usar y también mantenible. Sin embargo esto no quiere decir que deben descartarse características o que el software tenga errores.
- Tiene que ser sencillo pero **bien codificado**, es decir cumpliendo su objetivo y con calidad

Ej: en lugar de implementar una solución con múltiples capas de abstracción innecesarias, se opta por un diseño directo y claro, evitando sobrecargar el sistema.

«La simplicidad es siempre mejor que la funcionalidad».

Peter Hintiens



3-Siete principios de la ingeniería de software (2)

3er principio: Mantener la visión.

- Tener **la visión clara** es fundamental en el desarrollo de un software, conocer a donde se tiene que llegar a dar las pautas necesarias para tener un concepto del proyecto.
- Tener un arquitecto que pueda mantener la visión y que obligue a su cumplimiento garantiza un proyecto de software muy exitoso.

Ej; En un proyecto de desarrollo de software para una empresa de logística, todas las decisiones de diseño y funcionalidad se alinean con el objetivo de optimizar la gestión de envíos.

4to principio: Otros consumirán lo que usted produce.

- Al desarrollarse un software, se tiene que tomar en cuenta **que alguien será el usuario o varias personas serán las que ocupen el producto**, además habrá alguien que lo documente, que depure el código o le de mantenimiento e incluso ampliar el sistema.
- Por tal motivo como desarrolladores tenemos que construir el sistema pensando en la audiencia.

Ej: Un equipo de desarrollo documenta claramente su código y proporciona una API bien estructurada para que otros desarrolladores puedan utilizarla fácilmente en sus propios proyectos.

3-Siete principios de la ingeniería de software (3)

5to principio: Abrace al futuro.

- **El avance tecnológico tanto en hardware como en software se da en meses ya no en años**, por lo cual un dispositivo queda obsoleto al poco tiempo. El software tiene un tiempo de vida útil lo cual le da valor, sin embargo, **pierde este valor por los cambios que se dan al quedar obsoletas plataformas de hardware** u otros elementos.
- Los sistemas que lo logran son los que se mantienen y que fueron diseñados para eso, para ello está el principio “Nunca diseñes sobre algo iniciado”.

Ej: Al diseñar un sistema de autenticación, se opta por estándares modernos como OAuth 2.0, anticipando que será más seguro y adaptable a futuros cambios tecnológicos.

6to principio: Planee por anticipado la reutilización.

- La reutilización ahorra tiempo y esfuerzo, pero no hay que llevarlo al extremo puesto que no puede cumplir con los requerimientos del proyecto principal.
- **Reutilizar código es beneficios, pero es un reto tener un alto nivel del mismo.**
Para esto es necesario planificación y reflexión, utilizando técnicas adecuadas.

Ej: Se desarrolla un módulo de procesamiento de pagos que puede integrarse en diferentes aplicaciones sin necesidad de reescribir el código.

3-Siete principios de la ingeniería de software (4)

7mo principio: Pensar

- Al pensar en hacerlo **es probable que se haga bien**, y si sale mal volver a pensar, esto proporciona conocimientos y experiencia. Debe ser equilibrado y pensar con intensidad aplicándolo a los principios antes mencionados.

Ej: Antes de cambiar la arquitectura de un sistema, el equipo considera las implicaciones en el rendimiento, la seguridad y la escalabilidad, evitando decisiones impulsivas.



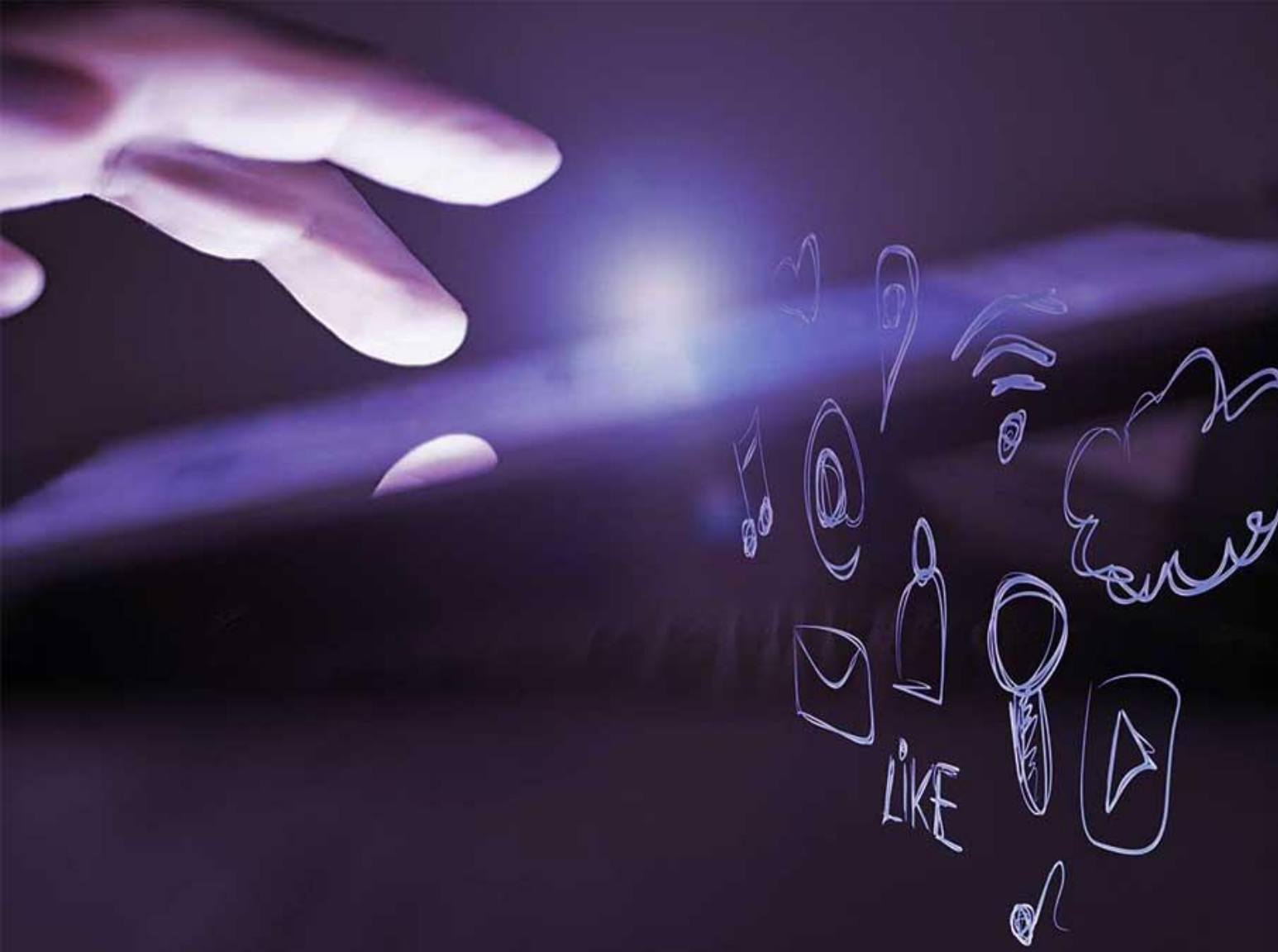
“Pensar en todo con claridad antes de emprender la acción casi siempre produce mejores resultados”

4-Clean Code

“Cualquier programador es capaz de escribir código que un computador puede entender.

Un buen programador es capaz de escribir código que un humano puede entender.”

Martin Fowler



4-Clean Code

Es posible implementar una misma funcionalidad escribiendo muy diferentes códigos. Si bien todos los códigos obtienen el mismo resultado, no todos los códigos son fácilmente mantenibles y extensibles (y por tanto testeables).

El **código limpio o sostenible** es aquel que **es fácil de entender por un humano y, por lo tanto, fácil de modificar, extender y probar en el tiempo.**

“El código sostenible es aquel que se puede mantener fácilmente a lo largo del tiempo. Para ello es necesario que su diseño sea intuitivo, que su complejidad sea la mínima imprescindible y que esté bien cubierto por pruebas automatizadas”

Carlos Ble, Código Sostenible (2022)



4-Clean Code o “Código Limpio”

Es un enfoque de programación que prioriza la claridad, la simplicidad y la facilidad de mantenimiento del código. Se trata de escribir código que funcione correctamente y sea comprensible para otros desarrolladores, e incluso para uno mismo en el futuro.

Un código limpio **evita complejidades innecesarias** y se organiza de manera que su propósito sea evidente al leerlo. Esto **implica nombrar variables y funciones de forma descriptiva, mantener funciones cortas y enfocadas en una sola tarea, y estructurar el código de manera lógica y coherente.**

Su valor radica en **que reduce la probabilidad de errores y facilita la colaboración en proyectos de software.** Un código claro y bien organizado es más fácil de modificar, actualizar y depurar, lo que ahorra tiempo y recursos a largo plazo. Además, fomenta la creación de software más robusto y adaptable a cambios futuros.



Codigo Limpio: buenas practicas	Descripción	Ejemplo
La legibilidad es clave	Un código claro y comprensible facilita el mantenimiento y el trabajo en equipo.	Usar nombres descriptivos en variables y funciones para que su propósito sea evidente sin comentarios adicionales.
Funciones pequeñas y específicas	Las funciones deben realizar una sola tarea bien definida para mejorar su reutilización y prueba.	Dividir una función compleja en varias funciones más pequeñas con responsabilidades específicas.
Nombres significativos	Los nombres de variables, funciones y clases deben reflejar su propósito.	En lugar de XW19, usar cantidadProductosVendidos para mejorar la claridad.
Eliminar código muerto	Eliminar código innecesario o no utilizado para reducir la complejidad y evitar confusiones.	Eliminar funciones obsoletas que ya no se llaman en el sistema.
Principio de la menor sorpresa	El código debe comportarse de manera predecible y seguir convenciones establecidas.	Respetar los estándares de nombres y estructuras comunes en el lenguaje de programación utilizado.
Modularidad y encapsulamiento	Organizar el código en componentes independientes y proteger detalles internos.	Crear módulos separados para lógica de negocio y acceso a datos, evitando la dependencia directa.
Escribir pruebas	Las pruebas unitarias garantizan que el código funciona correctamente y previenen errores.	Implementar pruebas automáticas para verificar la funcionalidad de cada componente antes de su despliegue.
Refactorización continua	Mejorar el código sin alterar su comportamiento externo para mantener la calidad a largo plazo.	Simplificar una función compleja eliminando redundancias y mejorando su estructura.

Reglas relacionadas a Código Limpio-1

Regla del noventa-noventa

“El primer 90% del código ocupa el 90% del tiempo de desarrollo. El 10% restante del código ocupa el otro 90% de tiempo de desarrollo.”

También se enuncia como: “el tiempo que falta para acabar el proyecto es constante”. La regla del noventa-noventa es una instancia del Principio de Pareto.

La Navaja de Oakham

Es una idea muy común, que llegó a la programación desde la filosofía. El principio obtuvo su nombre del monje inglés Guillermo de Oakham.

Este principio dice: «Las entidades no deben multiplicarse sin necesidad». En ingeniería, este principio se interpreta así: **no hay que crear entidades innecesarias sin necesidad.** Es buena idea pensar primero en los beneficios de añadir otro método/clase/herramienta/proceso, etc. Al fin y al cabo, si se añade otro método/clase/herramienta/proceso, etc. y no se obtiene ninguna ventaja más que el aumento de la complejidad, ¿qué sentido tiene?

Reglas relacionadas a Código Limpio-2

No lo vas a necesitar (YAGNI=You Aren't Gonna Need It)

Cuando un desarrollador añade todos los métodos posibles a la clase desde el principio y los implementa, e incluso puede no utilizarlos nunca en el futuro.

Por lo tanto, según esta recomendación, **primero implementa sólo lo que necesites, y más tarde, si es necesario, extiende la funcionalidad**. De esta manera, ahorrará esfuerzo, tiempo y nervios en depurar el código que no es realmente necesario.

Principio del menor asombro

Este principio significa que su **código debe ser intuitivo y obvio**, y no sorprender a otro desarrollador cuando revise el código.

Además, debes intentar evitar los efectos secundarios y documentarlos si no puedes evitarlos.

Diseño grande por adelantado

Antes de empezar a desarrollar la funcionalidad, debería **pensar primero en la arquitectura de la aplicación y diseñar todo el sistema** hasta detalles suficientemente pequeños, y sólo entonces proceder a la implementación según un plan predefinido.

Ventaja: el diseño correcto, es posible reducir considerablemente el costo de la depuración posterior y la corrección de errores. Además, tales sistemas de información, por regla general, son más escuetos y arquitectónicamente correctos.



Desventaja: la obsolescencia del plan durante el diseño y la elaboración. En este sentido, es necesario hacer cambios posteriores.



No te repitas (DRY= Don't Repeat Yourself)

Es un principio bastante simple pero muy útil que dice que **repetir lo mismo en diferentes lugares es una mala idea**.

- En primer lugar, está relacionado con la **necesidad de apoyo y modificación posterior del código**.
- Si algún fragmento de código se duplica en varios lugares dentro de un programa, hay una alta probabilidad de que se produzcan dos situaciones de alto impacto:
 - Al hacer incluso pequeños cambios en el código fuente, es necesario cambiar el mismo código en varios lugares. Esto requerirá tiempo, esfuerzo y atención adicionales (a menudo no es fácil).
 - Cualquier desarrollador puede pasar por alto accidentalmente uno de los cambios (puede ocurrir simplemente al fusionar ramas al versionar) y enfrentarse a los bugs.

Recomendación: si cualquier código se encuentra en el listado más de dos veces, debe ser colocado en una forma separada. Deberías pensar en crear un método separado incluso si encuentras una repetición por segunda vez.

Evitar la optimización prematura

La optimización es un proceso muy correcto y necesario para acelerar el programa, así como para reducir el consumo de recursos del sistema.

Pero todo tiene su tiempo. Si la optimización se lleva a cabo en las primeras etapas de desarrollo, puede hacer más daño que bien.

- ✦ Un código optimizado requiere más tiempo y esfuerzo para el desarrollo y el soporte. En este caso, a menudo hay que comprobar la **corrección del enfoque de desarrollo elegido al principio**.
- ✦ Al principio es más rentable **utilizar un enfoque sencillo pero no el más óptimo**. Y más adelante, al estimar cuánto ralentiza este enfoque el trabajo de una aplicación, pasar a un algoritmo más rápido o que consuma menos recursos.
- ✦ Además, mientras implementes inicialmente el algoritmo más óptimo, los requisitos pueden cambiar y el código irá a la basura. Así que **no hay necesidad de perder el tiempo en una optimización prematura**.

Ley de Demeter

La idea básica de este principio es **dividir las áreas de responsabilidad entre las clases y encapsular la lógica dentro de una clase, método o estructura.**

De este principio se pueden distinguir varias recomendaciones:

- 1) **Las clases o entidades deben ser independientes**
- 2) Se debe intentar **reducir el número de conexiones entre las diferentes clases** (el llamado **acoplamiento**).
- 3) **Las clases asociadas deben estar en un solo módulo/paquete/directorio** (también conocido como **cohesión**).

Siguiendo esos principios, **la aplicación se vuelve más flexible, comprensible y fácil de mantener.**

S.O.L.I.D.

Entre los objetivos de tener en cuenta estos principios a la hora de escribir código encontramos:

- **Crear un software eficaz:** que cumpla con su cometido y que sea **robusto y estable**.
- Escribir un **código limpio y** flexible ante los cambios: que se pueda modificar fácilmente según necesidad, que sea **reutilizable y mantenible**.
- **Permitir escalabilidad:** que acepte ser ampliado con nuevas funcionalidades de manera ágil.

⇒ En definitiva, **desarrollar un software de calidad**.

En este sentido la aplicación de los principios SOLID está muy relacionada con la comprensión y el uso de patrones de diseño, que nos permitirán **mantener una alta cohesión y, por tanto, un bajo acoplamiento de software**.

Reglas relacionadas a Código Limpio-7

S.O.L.I.D. es un grupo de principios de diseño orientado a objetos.

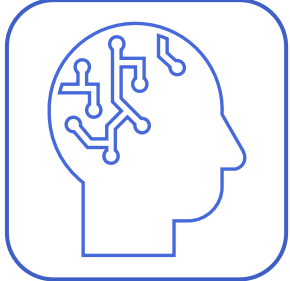
S	Single Responsibility Principle (SRP)	Las clases de alto nivel no deben depender de las clases de bajo nivel; ambas deben depender de abstracciones.-
O	Open/Closed Principle (OCP)	Las entidades de software deben estar abiertas para extensión, pero cerradas para modificación. La apertura-cerrado establece que las entidades de software (clases, módulos, funciones, etc.) deben estar abiertas a la extensión, pero cerradas a la modificación. Este principio es a la hora de desarrollar clases, librerías o frameworks.
L	Liskov Substitution Principle (LSP)	La sustitución de Liskov establece que la clase heredada debe complementar, no reemplazar, el comportamiento de la clase base
I	Interface Segregation Principle (ISP)	No se deben forzar las interfaces a implementar métodos que no necesitan. La segregación de la interfaz establece que ningún cliente debe ser forzado a depender de métodos que no utiliza
D	Dependency Inversion Principle (DIP)	Las clases de alto nivel no deben depender de las clases de bajo nivel. La inversión de la dependencia dice que el programador debe trabajar a nivel de la interfaz y no a nivel de la implementación.

Clean Code & SOLID

Juntos forman un marco integral de buenas prácticas:
Clean Code cuida el detalle, **SOLID** cuida la estructura



Clean Code asegura que cada línea y función sea clara y mantenible.



SOLID asegura que la arquitectura del sistema sea flexible y escalable.



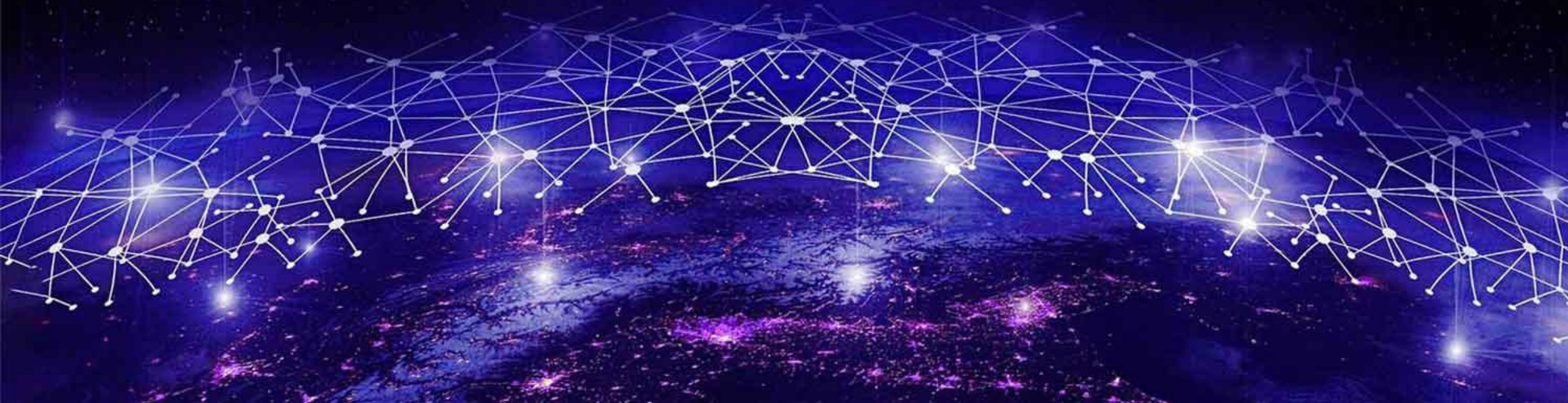
Aspecto	Clean Code	SOLID
Definición	Principios para escribir código claro, legible y mantenible.	Conjunto de 5 principios de diseño orientado a objetos.
Objetivo central	Legibilidad y mantenibilidad inmediata.	Flexibilidad, extensibilidad y bajo acoplamiento a largo plazo.
Alcance	Micro: estilo, funciones, nombres, comentarios, simplicidad.	Macro: arquitectura, relaciones entre clases, interfaces y dependencias.
Aplicación típica	Refactorización de funciones, claridad en nombres, evitar duplicación.	Diseño de clases con SRP , uso de interfaces, inversión de dependencias.
Beneficios	- Reduce errores humanos y facilita colaboración en equipos. - Facilita comprensión rápida del código por cualquier desarrollador.	- Asegura evolución sostenible del software en proyectos grandes. - Permite escalar y modificar sistemas sin romper otras partes.
Nivel de aplicación	Micro: funciones, clases, nombres.	Macro: arquitectura, relaciones entre clases.
Riesgo si se ignora	Código “espagueti”, difícil de leer y mantener.	Arquitectura rígida, difícil de escalar o adaptar.



Cuando se aplican juntos, estos principios **ayudan a un desarrollador a crear un código que es fácil de mantener y extender en el tiempo.**

Las buenas prácticas que pueden ayudar a escribir un mejor código: **más limpio, mantenible y escalable**

Son heurísticos, basados en la experiencia: “se ha observado que funcionan en muchos casos; pero no hay pruebas de que siempre funcionen, ni de que siempre se deban seguir.”



4-Spagueti Code

El término es una metáfora: *“al igual que un plato de espaguetis, donde los fideos se entrelazan en un desorden desordenado, el código espagueti es una masa caótica de lógica interdependiente, bucles y saltos”* .

- Trabajar con este tipo de código puede resultar frustrante, especialmente cuando es necesario realizar cambios, implementar mejoras o corregir errores.

Código Espagueti: mal estructurado, complicado y difícil de leer, comprender y mantener

El código espagueti se refiere a una estructura de código desorganizada y difícil de seguir, donde la lógica se retuerce de forma impredecible, lo que dificulta su comprensión y mantenimiento.



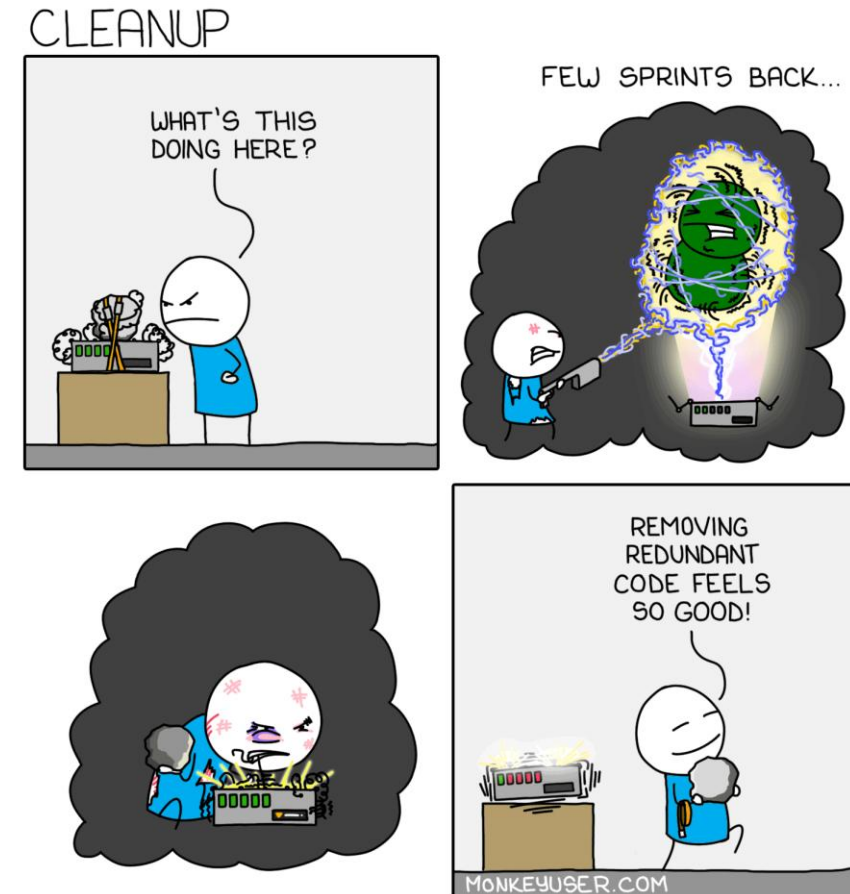
Es un problema persona y los alcances y responsabilidades de su rol en el desarrollo de un sistema, que aplica codificación para desarrollar sus tareas y objetivos.

- ✳ **Desarrolladores** que diseñan, maquetan y programan las interfaces y lógica en el Front-end (UI/UX, frameworks , librerías), Back-end, diseño del modelo de datos, la DB y su optimización, adaptación a distintos dispositivos (móviles, tablets, colectores de datos, etc.), manejo de APIs, optimización del rendimiento, implementación de seguridad, bug-fixing, refactorización, implementación de la arquitectura, mantenimiento, cómo se hace la gestión de configuración del código, integración con herramientas y servicios externos, monitoreo de métricas, migración de código heredado, etc..
- ✳ **QA engineers** cuando programa frameworks de automatización de testing, escribe y corre scripts de pruebas automatizadas, usa mocking y stubbing para simular servicios externos en pruebas, genera API testing, automatiza reportes de errores y métricas, diseña sistemas para generar datos de pruebas, escribe pruebas de rendimiento, implementa pruebas de seguridad, agrega pruebas en pipelines CI/CD
- ✳ **DevOps** cuando automatizan scripts o pruebas de seguridad, gestionar la infraestructura en el cloud, generan scripts para orquestar contenedores, CI/CD, generar métricas, etc.

Código Espagueti: cómo y porqué está escrito mal?

No está asociado a un lenguaje o entorno en particular, el problema es **“COMO ESTÁ ESCRITO EL CODIGO”**, surgido de una o más **situaciones** como por ejemplo:

1. **Desarrollo rápido o bajo presión**, improvisando y sacrifican la calidad del código por la velocidad, llevando a **soluciones rápidas y hacks (soluciones rápidas y sucias)**.
2. **Falta de experiencia**, diseñando código no escalable ni adaptable.
3. **Añadir nuevas funciones sin refactorizar**, puede generar dependencias complejas y una lógica difícil de gestionar.
4. **Falta de planificación**, codificar sin haber pensado en la arquitectura ni diseño.
5. **Mala comunicación en los equipos**, la falta de comunicación o de prácticas de codificación estandarizadas, puede resultar en bases de código inconsistentes y desordenadas.
6. **Legacy Code o Código Heredado**: el código se escribió para abordar algún caso excepcional que no se conocía durante el diseño/desarrollo original.



Código Espagueti: malos olores...

Code Smell

CODE SMELLS ARE
SYMPTOMS OF POOR
DESIGN OR
IMPLEMENTATION CHOICES

[Martin Fowler]



El código es difícil de mantener, extender y probar desprende “malos olores o no huele bien” (code smell). Esto nos permite identificarlo y es el momento de refactorizarlo.

Algunos de los más habituales en la **POO** son:

- ❖ **Métodos o clases muy grandes.** Es un síntoma de falta de cohesión. Un método con muchas líneas de código suele tener demasiadas responsabilidades.
- ❖ **Obsesión por los tipos primitivos.** Los únicos tipos utilizados son los que ofrece el lenguaje de forma nativa (Integer, String, Boolean). No se construyen tipos específicos (clases) como Nombre, Teléfono o DNI.
- ❖ **Métodos con muchos parámetros de entrada.** Es un síntoma de falta de cohesión. Un método que requiere muchos parámetros suele tener demasiadas responsabilidades.
- ❖ **Clases sin métodos.** Las clases son usadas para estructurar datos, no para modelar el negocio. El código que opera con esos datos es “desperdigado” por otras clases.
- ❖ **Abuso de variables globales.** Los diferentes componentes del software hacen uso de las mismas variables globales para persistir el estado del software o comunicar eventos.

Características comunes del Código espagueti

Funciones largas y monolíticas

Falta de control

Lógica duplicada

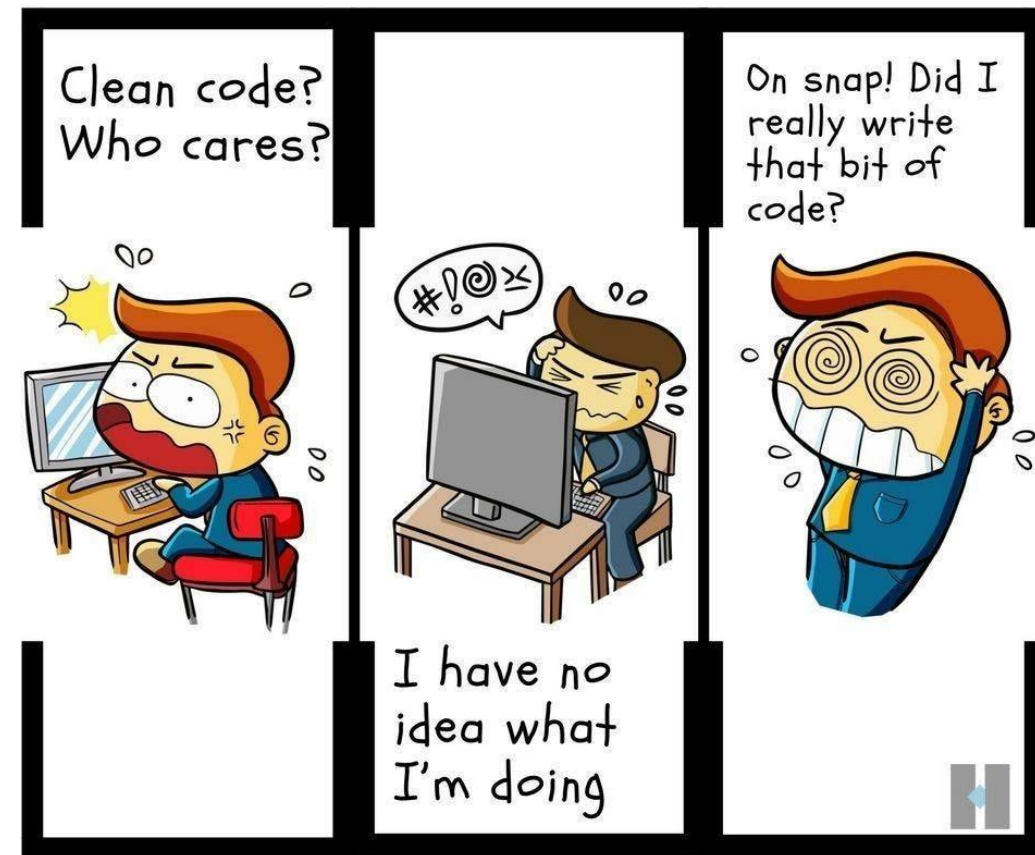
Exceso de variables globales

Ej.: en funciones masivas, accesos del usuario a BD y la gestión de errores

Ej.: bucles excesivos, condiciones, If anidados, Go to, etc.

Ej.: no reutiliza funciones o métodos, suele tener la misma lógica, o una similar, dispersa en varios lugares. Por lo cual, hay que rastrear cada ocurrencia de un error, un proceso agotador y propenso a errores.

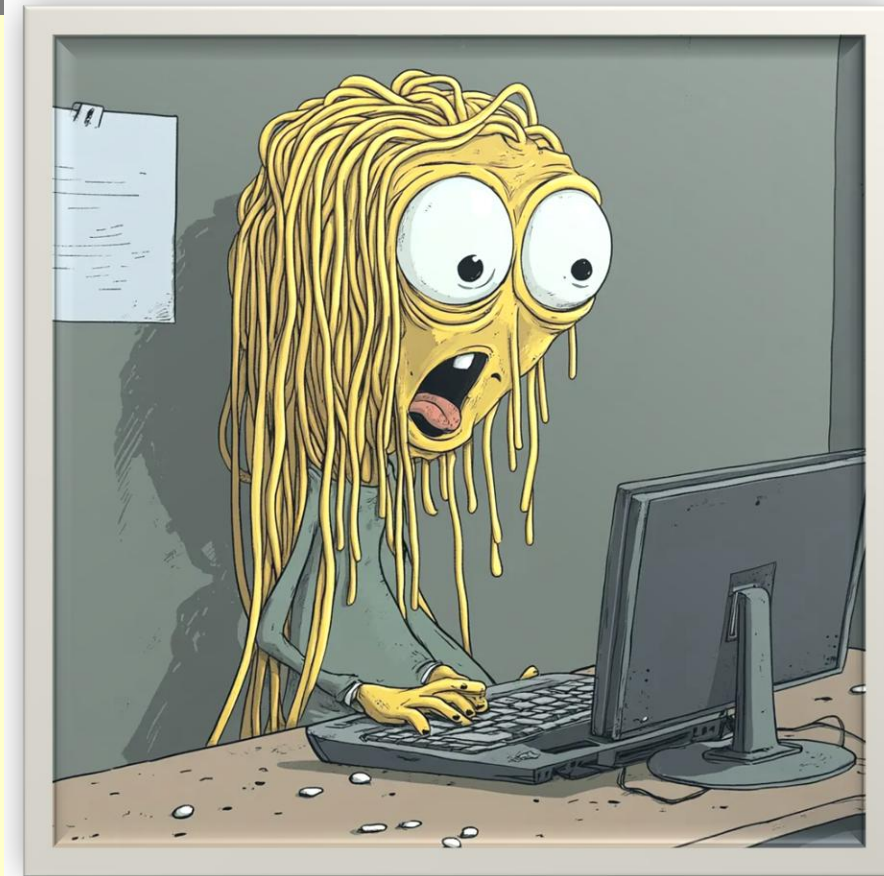
Ej: variables globales que se pasan y modifican desde todas partes. Es como intentar encontrar un fideo en un plato de pasta.



Código espagueti: peligros

El peligro del código espagueti es que provoque:

- ⚠ **Mantenimiento difícil**, requiere de inmersión en el tiempo para añadir funciones, depurar o corregir errores, entender que pasa o actualizar código complejo requiriendo de mucho tiempo
- ⚠ **Alto riesgo de errores** y que los cambios facilitan la introducción de errores al modificar el código.
- ⚠ **Baja escalabilidad**, no puede crecer ni evolucionar, requiriendo reescrituras exhaustivas.
- ⚠ **Dificultad para la colaboración en equipo**
- ⚠ **Productividad Reducida** Los desarrolladores dedican más tiempo a descifrar el código que a implementar nuevas funciones.
- ⚠ **Mayor Deuda Técnica** Con el tiempo, el coste de mantener y corregir código espagueti supera el tiempo inicial ahorrado.



Soluciones para el código espagueti

Estrategias comprobadas para prevenir y corregir el código espagueti:

1. Seguir las mejores prácticas

- Adoptar estándares y directrices de codificación (Ej. PHP: PSR, Java: Java Code Conventions)
- Utilizar convenciones de: Nomenclatura (Ej. En Java: clases en PascalCase, métodos y variables en camelCase), Indentación (4 espacios), Comentarios (Ej: Java: usar Javadoc `/**...*/`, en .Net: `///` para XML), Componentes (Ej. En ReactJS: uso de funciones en lugar de clases),

2. Aprovechar patrones establecidos

- Usar MVC, Singleton o Factory para estructurar el código de forma lógica.

3. Refactorizar periódicamente

- Para limpiar, clarificar, estructurar, simplificar y eliminar la lógica redundante, sin cambiar su funcionalidad.

4. Utilizar el control de versiones

- Para garantizar el seguimiento de los cambios, lo que facilita la reversión a un estado limpio.

5. Priorizar las revisiones de código

- Las revisiones por pares ayudan a identificar y solucionar posibles problemas de forma temprana, mejorando la calidad general del código.

6. Aprovechar la programación modular

- Dividir el código en módulos pequeños, reutilizables e independientes para mejorar la legibilidad y el mantenimiento.

7. Invertir en capacitación

- Capacitar a tu equipo sobre las mejores prácticas, los principios de diseño y la importancia de una codificación limpia.

Herramientas de ayuda al desarrollo: lower case	
La industria IT crea herramientas software para construir software. Son las llamadas herramientas lower CASE	
Tipos de Lower Case	Algunos Ejemplos
Entorno de Desarrollo Integrado (IDE) (tool para escribir y gestionar el código)	Intellij, Visual Studio, NetBeans, Pycharm, Eclipse, Xcode, Android Studio, Dreamwaver, Codepen, Next.JS, Svelte
Depuradores	Visual Studio Debugger, PDB, Chrome Devtools
Control de versions	GitHub, GitLab, Bitbucket, Mercurial, Git
Integración Continua	GitHub Actions, GitLab CI/CD, CircleCI, Jenkins, Travis CI
Frameworks (base estructurada para codificar) y Librerías	React, Angular, Laravel, Django, Spring Boot, FastApi, Next.JS, Svelte
Analizadores de Código (revisan Calidad)	SonarQube, DeepCode, Codiga, Checkstyle, Eslint, PyLint
Autogeneradores (crea código a partir de reglas o plantillas predefinidas)	Yeoman, Jhipster, Swagger, Rails generators, Yeoman,
Automatización, optimización, despliegue y contenedores	Docker, Gradle, Make, Terraform, GitHub Actions, OpenAI, Nvidia, Firefly Bulk
Base de datos	Supabase, Cockroachdb, Duckdb, Redis
Asistentes de codificación (crean código)	GitHub Copilot, Amazon Codewhisperer, Tabnine
Plataformas de Machine Learning	Tensorflow, Pytorch, Jax

4-Ingeniería de software como disciplina profesional

- Un principio es una ley importante y es requerida en un sistema de pensamiento.

La ingeniería del *software*, vista como una profesión, es bastante joven comparada con las ingenierías tradicionales y tiene aún mucho que aprender de ellas. Existen aspectos que caracterizan a las ingenierías tradicionales que todavía no han sido dominados por la ingeniería del *software*; es decir, no se practican con la rigurosidad que una ingeniería establecida requiere.

Denning y Riehle describen seis de estos aspectos:

- 1) predictibilidad de los resultados,
 - 2) uso de métricas de diseño,
 - 3) tolerancia a fallas,
 - 4) separación entre diseño e implementación,
 - 5) reconciliación entre restricciones o requisitos conflictivos
 - 6) adaptación a ambientes cambiantes.
- Estos aspectos han ido cambiando en los últimos años gracias a los avances de la ingeniería del *software* como disciplina establecida.



Ingeniería de software como disciplina profesional

- La **predictibilidad de los resultados** del proceso de ingeniería ha sido abordado por la ingeniería del *software* a través de: **aplicación de los procesos de gestión de proyectos** y el uso de una amplia variedad de métodos ágiles y disciplinados que existen actualmente y cuya aplicación al desarrollo de *software* permiten predecir el resultado de un proyecto de desarrollo de *software*.
- Una profesión consolidada tiene **cuatro elementos o pilares** sobre los cuales se apoya:
 1. Un cuerpo de conocimientos
 2. Un código de ética
 3. Modelos curriculares
 4. Programas de certificación profesional



Malas prácticas de Desarrollo y Mantenimiento

- ⊗ Planificación y estimaciones imprecisas, o no se planifica
- ⊗ No se recopilan datos de proyectos pasados.
- ⊗ Se invierte más dinero en mantenimiento que en formación de los ingenieros en las nuevas tecnologías de desarrollo.
- ⊗ No se documenta lo suficiente.
- ⊗ Se pasa directamente a la codificación.
- ⊗ Procesos software improvisados
- ⊗ No se siguen rigurosamente las especificaciones
- ⊗ No se hace planificación de riesgos.
- ⊗ Se resuelven crisis inmediatas, apagando fuego
- ⊗ Se sacrifica funcionalidad y calidad del producto para cumplir plazos.
- ⊗ No se realizan pruebas, verificaciones o revisiones del SW



- ⇒ Baja calidad del software desarrollado
- ⇒ Alto grado de desconfianza e insatisfacción en el cliente
- ⇒ Empresas inmaduras o proyectos artesanales
- ⇒ Se exceden en los plazos y presupuestos previstos
 - 90% de los proyectos no consiguen los objetivos propuestos
 - 40% fracasan completamente
 - 29% nunca se entregan

Por qué es Complejo el software?

Brooks: “La complejidad del Software es una propiedad esencial y no accidental”

Complejidad Accidental

Se debe a la **manera en que intentamos solucionar el problema**

Complejidad Esencial

Es **inherente al problema** en sí mismo.

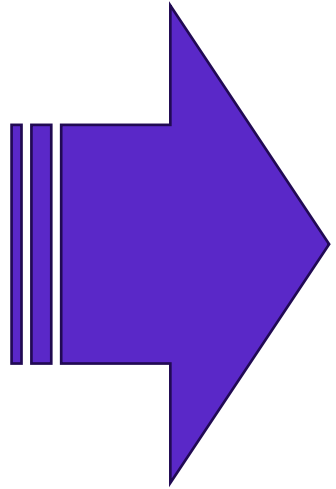
MOTIVOS

La complejidad del dominio del problema

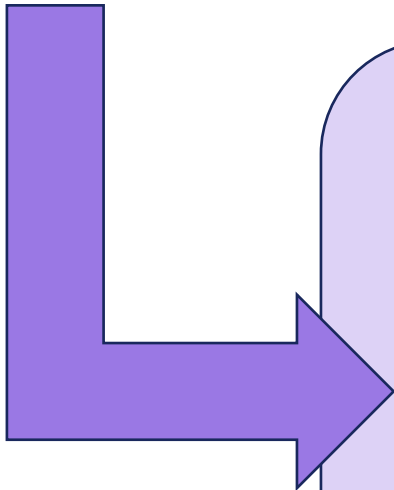
La dificultad de controlar el proceso de desarrollo

Los problemas para caracterizar sistemas discretos

Propuesta



- Aplicar métodos, técnicas y herramientas de desarrollo
- Adoptar estándares de desarrollo
- Utilizar la experiencia acumulada
- Documentación
- Aplicar estándares, lecciones aprendidas y buenas prácticas



- ⇒ Mejorar el proceso
- ⇒ Reducción de costes
- ⇒ Reducción del tiempo de desarrollo
- ⇒ Reducción de riesgos
- ⇒ Mejora de la calidad del producto
- ⇒ Protección del cliente
- ⇒ Protección de la organización
- ⇒ Aumenta su competitividad y productividad

5-Desarrollo Global de software (DGS)

Desarrollo Distribuido de Sw

Los stakeholders que llevan a cabo los proyectos se **encuentran distribuidos entre varios sitios remotos**.

Son sub-equipos y equipos **en lugares geográficos diferentes, haciendo uso de las tecnologías de comunicación** para interactuar y llevar a cabo una tarea en conjunto.

Desarrollo Global de Sw

Los equipos se distribuyen mas allá de la frontera que deben hacer frente a desafíos únicos y de gran interesa para la comunidad informática.

Los equipos pertenecen a distintos sub-equipos de la misma organización

Offshoring: deslocalización, transferencia de empleos a otro país.

Stakeholders: clientes, usuarios, desarrolladores, líderes, expertos en el dominio, etc.

Modelo de Global Delivery

- Hace referencia a una estrategia de realizar el delivery de los procesos de negocio, por su naturaleza, se podrán estar realizando **usando varias modalidades simultáneamente**.
- Este modelo se basa en una **estrategia horaria**, es decir, contando con oficinas a lo largo del planeta, para dar soporte o tener un centro que funciona las 24 horas
- Esto se consigue con el **bajo costo de las comunicaciones**, y teniendo la combinación óptima de los modelos bajo el **mismo gerenciamiento y usando los mismos procesos**.



Desarrollo Global de Software

ESCENARIO	Misma organización	Diferente Organización
Mismo lugar	Desarrollo co-localizado	Desarrollo co-localizado con subcontratación
Mismo país	Desarrollo distribuido (DSD)	Desarrollo distribuido con subcontratación (DSD)
Otro país	Deslocalización Desarrollo Global (GSD)	Subcontratación deslocalizada Desarrollo Global (GSD)

Clasificación del trabajo de acuerdo a la intensidad de comunicación



6-Desarrollo Global de software: Beneficios y Problemas

Beneficios DGS

Aprovechar la diferencia horaria para lograr jornadas laborales más largas y conseguir más productividad –follow the sun–

Minimizar los costes de desarrollo. Según lo que externalice: CF a CV

Localizar a los desarrolladores más cerca del cliente

Obtener ventajas de la diversidad de experiencias, conocimiento técnico y destrezas de los stakeholders distribuidos (a favor de la innovación y compartiendo mejoras prácticas y se mejora el proceso)

Optimizar los procesos del negocio y aumentar productividad

Problemas DGS

Ocasionados por la comunicación inadecuada

Ocasionados por la diversidad cultural - idioma

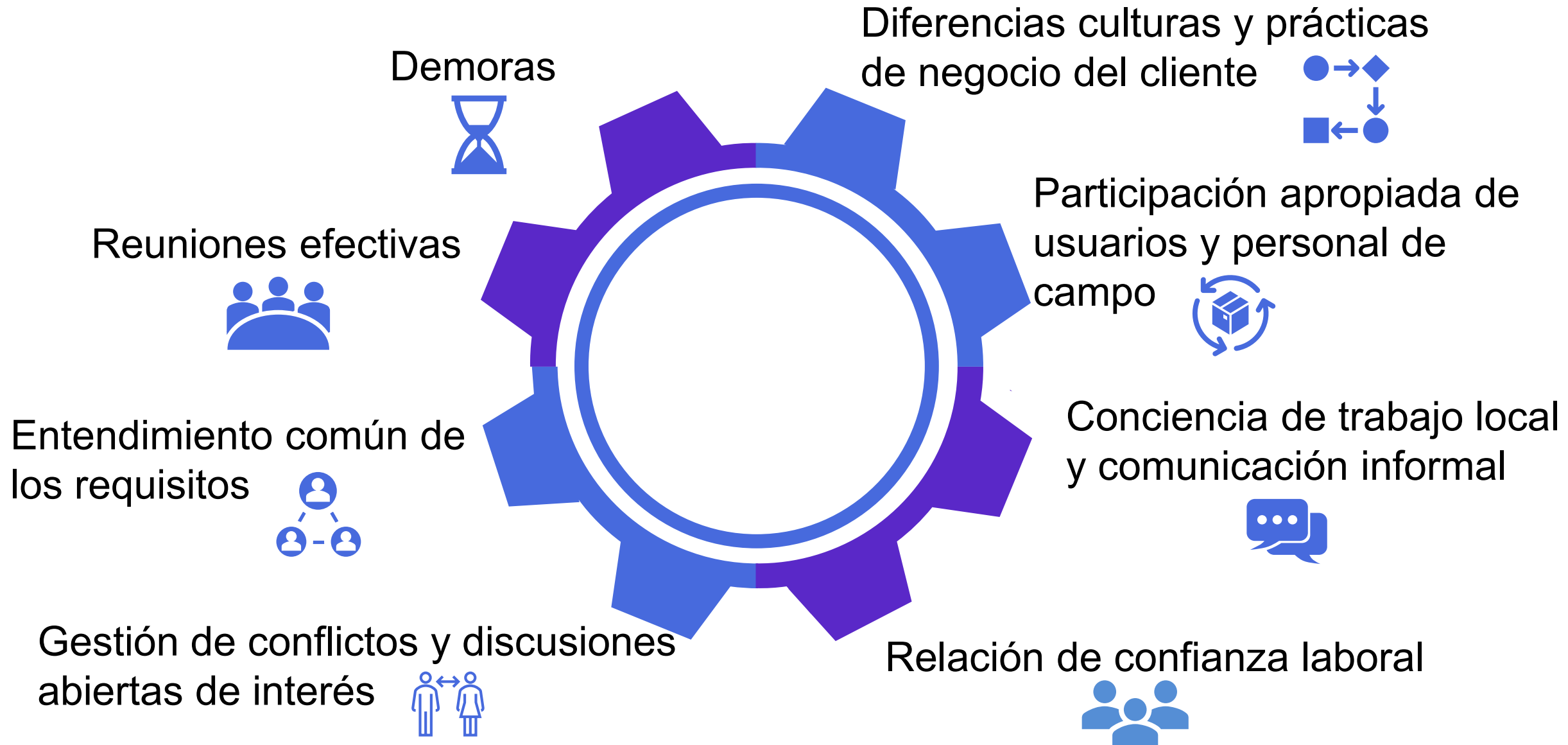
En la gestión del conocimiento

Ocasionados por la diferencia horaria

Dependencia de la tecnología de comunicación

Vulnerabilidad de la seguridad de datos

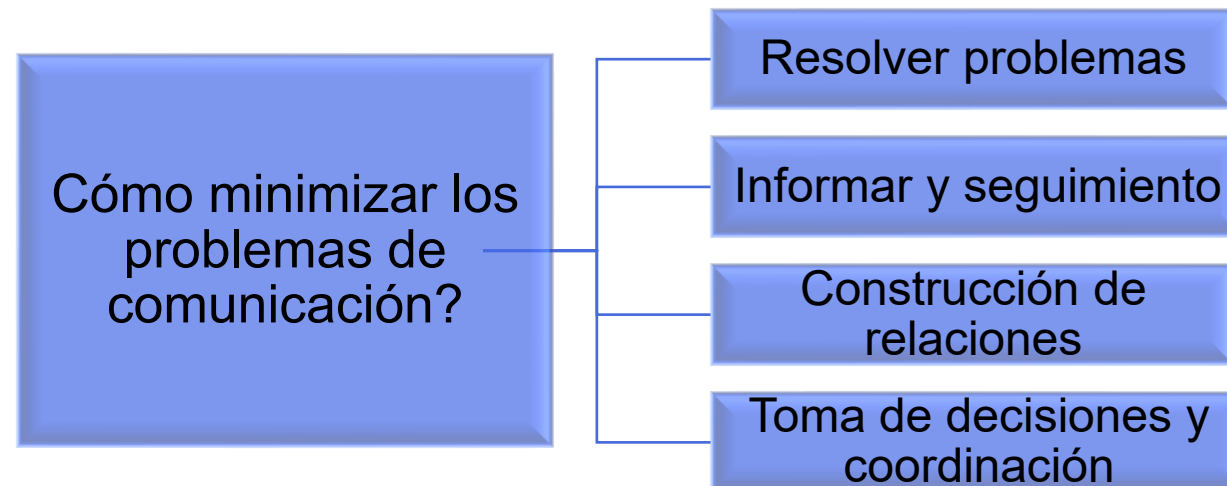
6-Desarrollo Global de software: desafíos identificados



Herramientas de apoyo al trabajo en grupo distribuido

Espacio/ Tiempo	Mismo Momento	Diferentes Momentos
Mismo lugar	Interacción cara a cara	Interacción asincrónica
Distinto lugar	Interacción sincrónica distribuida	Interacción asincrónica distribuida

- **Herramientas asincrónicas**: e-mail, grupos de noticias y novedades, foros, boletines electrónicos, documentos compartidos, pizarras de dibujos compartidas asincrónicas, etc.
- **Herramientas sincrónicas**: pizarras de dibujo compartidas sincrónicas, chat, mensajería instantánea, conferencias, videoconferencia, etc.



7-Modelos de provisión y la globalización de servicios

- 
- Características más importantes donde hay **trabajo remoto**...
 - Dispersión geográfica, cultural y escala de valores
 - Diferentes culturas de empresas, husos horarios, idiomas, procesos de trabajo, entendimiento de roles, formas de trabajo, experiencias, visiones del producto/servicio, etc.
 - Team Building y Visibilidad
 - Comunicación efectiva y fase-to-face
 - Lograr confianza y buena relación con el cliente
 - La gente va primero, los procesos y la tecnología después...
 - Confianza y comunicación
 - Participación constante
 - Solución de conflictos

Outsourcing

Es un sistema de contratación en el que una empresa recurre a otra para realizar tareas especializadas.

- En este proceso, la organización confía la ejecución de ciertas actividades a proveedores externos que son expertos en su campo.
- Se subcontrata una empresa o profesionales para el cumplimiento de ciertas tareas o servicios internos, que de otro modo le resultaría más costoso (en tiempo o dinero) asumir.
- Si se usan múltiples empresas de 3eros para completar un proyecto de desarrollo de software se llama: **MULTISOURCING**

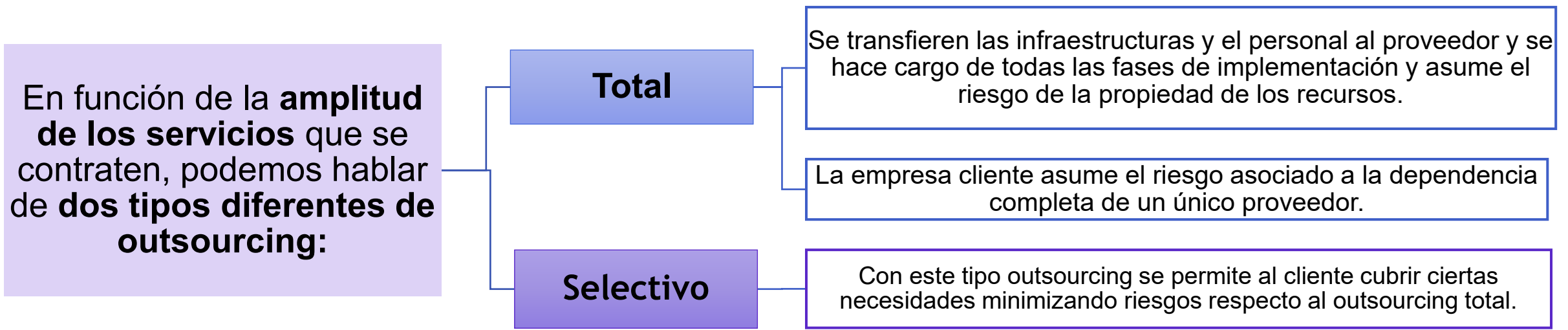


Outsourcing

- La razón más importante de recurrir al outsourcing es la de delegar la responsabilidad y preocupación de la administración operacional.
- Un punto clave, en toda empresa que quiera recurrir al **outsourcing**, **es evitar tener con un único proveedor**, para no ser dependientes de él.

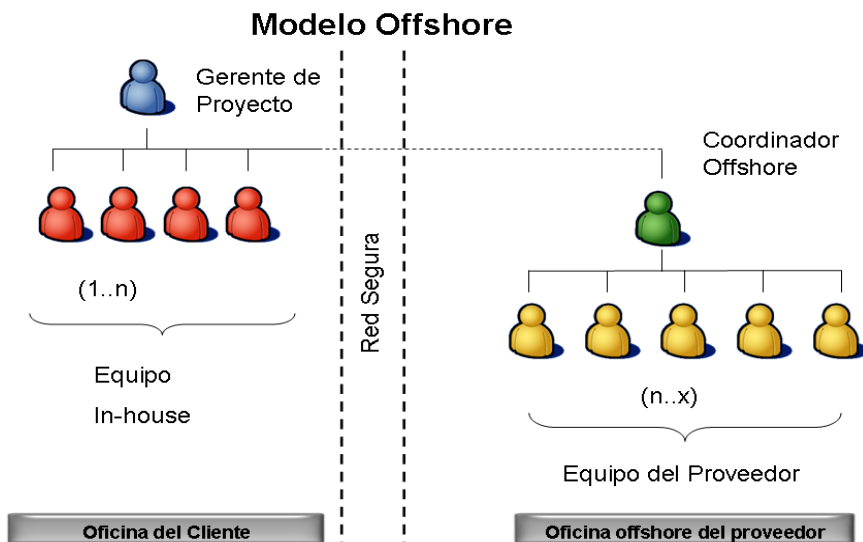
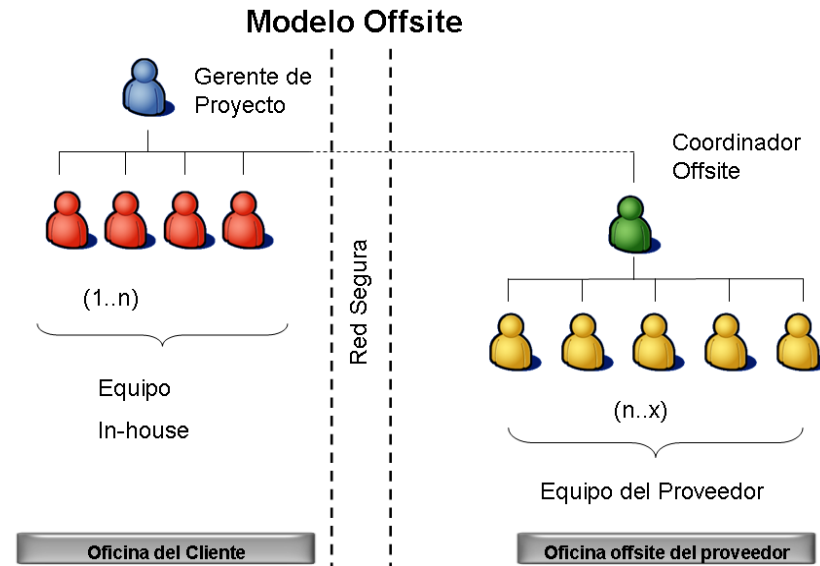
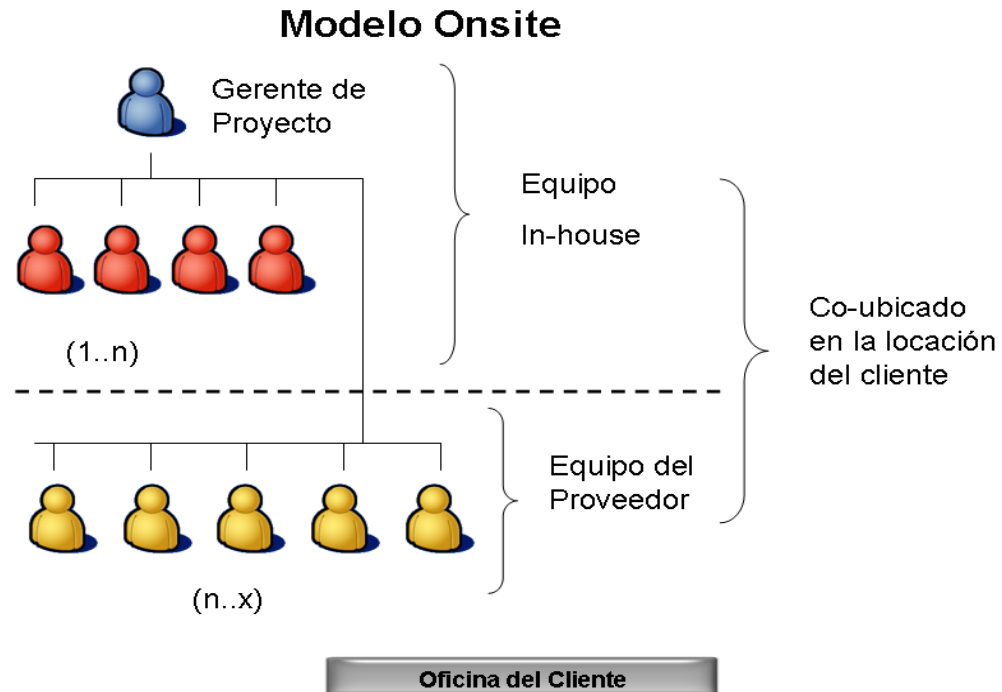
Se tiene que tener en cuenta:

- **Proveedor** con equipo compatible en software, hardware y procedimientos operativos.
- **Asegurar el acceso a datos y software propio.**
- Preparar un **plan de contingencia**.



Modelos de Delivery Global				
Término	Descripción	Ubicación Geográfica	Ventajas	Desventajas
ONSITE	Trabaja físicamente en las instalaciones del cliente durante un período determinado.		- Comunicación directa con el equipo de gestión. - Integración rápida con el personal existente.	- Puede generar tensiones entre contratistas y empleados permanentes. – - Mayor costo logístico.
ONSHORE	El trabajo se subcontrata a empresas dentro del mismo país.		- Zona horaria similar. - Facilita la comunicación y colaboración.	- Costos laborales más altos que offshore.
NEARSHORE	Se subcontrata en países vecinos o con zonas horarias similares.		- Menor diferencia horaria. - Facilita las visitas y la comunicación. - Mejores tarifas	- Costos + altos que offshore. - Limitación de talento: puede ser restringida por la ubicación geográfica
OFFSHORE	El trabajo se envía a empresas en otros países , generalmente con costos laborales más bajos.	En países diferentes.	- Reducción significativa de costos. - Acceso a talento global.	- Diferencia horaria: puede dificultar la comunicación. - Comunicación: barreras idiomáticas y diferencias culturales
OFFSITE	El personal de outsourcing trabaja fuera de las instalaciones del cliente.	Puede ser dentro o fuera del país.	- Flexibilidad en la ubicación y de adaptación al proyecto. - Menor costo logístico.	- Comunicación no presencial. - Menos control directo.
HÍBRIDO	Combina elementos de onshore y offshore. La gestión se realiza localmente, pero se contratan off- o nearshore para la mayor parte del trabajo.		- Flexibilidad: Permite adaptarse a las necesidades específicas del proyecto. - Equilibrio: Combina ventajas, como la comunicación fluida del onshore y las tarifas competitivas del offshore	- Requiere una gestión cuidadosa para equilibrar los equipos y procesos

Ejemplos gráficos de Outsourcing



OFFERING INTEGRADO **OUTSOURCING**

GOBIERNO DEL SERVICIO

Gestión del servicio, Metodologías KPI's, Conocimiento, Gestión del cambio, Mejora continua, Gestión de terceros, Innovación, Transformación.

SOPORTE A LA DISPONIBILIDAD Y EL DESEMPEÑO



MESA DE SERVICIO

Punto único de contacto para la atención y resolución de incidentes.



SOPORTE EN SITIO

Intervención física en el dispositivo y/o puesto de trabajo.



REDES PERIMETRALES

Soporte a la disponibilidad, desempeño de los servicios de conectividad.



CÓMPUTO CENTRALIZADO

Soporte de la disponibilidad, desempeño de los servicios de cómputo centralizado.



BASE DE DATOS

Soporte a la disponibilidad, desempeño de la DATA de los servicios.



CAPA MEDIA

Soporte a la disponibilidad, desempeño de los servicios de integración y aplicaciones.



APLICACIONES

Soporte a la disponibilidad, desempeño y evolución de aplicaciones (DevOps).



BPO / BPA

Ejecución de actividades propias o de soporte del negocio.

SERVICIOS TRANSVERSALES

HYBRID CLOUD

Gestión, organización y portabilidad para las cargas de trabajo en múltiples nubes.



MONITOREO / NOC

Visibilidad del estado de los componentes y servicios / primeras interacciones, escalafones y seguimiento.



DOC

Operación continua de los Centros de Datos.



HERRAMIENTAS

Potencian la entrega y cumplimiento de servicios.



SEGURIDAD

Protección y cumplimiento.



Modelos de Outsourcing de proyectos IT más populares

1

IT STAFF AUGMENTATION

El proveedor complementa el equipo existente del cliente proporcionando recursos adicionales especializados.

2

PROJECT-BASED MODEL

La empresa externaliza un proyecto o conjunto de tareas específicas.

3

EQUIPO DEDICADO DE DESARROLLO

Este modelo es adecuado para proyectos a largo plazo o cuando el cliente necesita desarrollo y soporte continuo.

4

SERVICIOS GESTIONADOS

El proveedor asume la responsabilidad total de gestionar y entregar servicios o funciones específicas de software.

Modelos de provisión y la globalización de servicios: herramientas

Las **herramientas** que más se usan son:

- Colaboración y mensajería (Slack, Discord, MS Teams)
- Control de código fuente (Git, Mercurial, SVN, Perforce)
- Herramientas de revisión de código (Github, Crucible, Review Board, Axolo, Collaborator)
- Administración de proyectos (Jira, Trello, Project Libre, Asana, etc.)
- Administración de despliegues (Gitlab, Kubernetes, Docker Swarm, Mesos, AWS ECS, EKS)
- Repositorios de documentación (SharePoint, Confluence, DocuWare, Adobe Doc Cloud, GitBook)
- Captura de requerimientos, análisis y diseño (Jama Sw, Doc Sheets, ReqSuite, Orcanos)
- Desarrollo y programación (Visual Studio, IntelliJ, Collaborator)
- De Evaluación y Pruebas (Apache JMeter, Testim, Katalon Studio, Postman, Allure TestOps, Mabl, Appium, Kobiton, Cypress, Selenium)
- Software de gestión de recursos humanos (SAP)
- Gestión de redes sociales (Hootsuite, Bandwatch) y plataformas profesionales (Linkedin, Universia, beBee)
- Sistema de reclutamiento (CATs, Linkedin Recruiter)
- Plataformas de video entrevista (Zoom, Meet, Skype, VidCruiter)
- Software de onboarding digital (Enboarder, Talmundo)
- A estas se suman nuevas herramientas que aplican Inteligencia Artificial

Como puede concluirse, todas estas herramientas requieren de una **red de comunicaciones segura y confiable.**